

NOURISHNEST

NourishNest Technical Project Documentation

AI assisted nutrition, pantry, recipe, meal planning, and grocery management

From Household Data to Actionable Planning

PROFILE	Households • members • preferences • allergies
KNOWLEDGE	Foods • USDA provenance • recipes • nutrition
STATE	Pantry locations • lots • expiration • low stock
PLANNING	Recommendations • weekly plan • requirements • shortages
ACTION	Grocery lists • purchases • optional pantry intake

Product capability map

Submission edition • Implementation status through Phase 9A • September 2026

Document Purpose

This report documents the complete NourishNest build for technical review and weekly demonstration. It explains the user problem, system architecture, database evolution, application workflows, agentic AI design, evaluation framework, safety controls, verification evidence, current limitations, and production roadmap.

NourishNest has progressed from a household grocery concept into a tested full-stack application. Its central design decision is to separate probabilistic language interpretation from deterministic calculations and transactional operations. The AI assistant can understand a planning request and coordinate approved read-only tools, while nutrition, inventory, recipe, and grocery results remain grounded in typed application services.

Contents

- 1 Executive Summary
- 2 Product Vision and User Problem
- 3 Functional Capabilities
- 4 System Architecture
- 5 Data Architecture and Migrations
- 6 Core Application Workflows
- 7 Agentic AI Architecture
- 8 Evaluation and Quality Engineering
- 9 Security and Reliability
- 10 User Experience
- 11 Phase History and Weekly Demonstration Guide
- 12 Operations and Configuration
- 13 Limitations and Roadmap
- Appendix A API Inventory
- Appendix B Demonstration Script

1 Executive Summary

NourishNest is an AI-assisted household food-management platform that connects household profiles, dietary needs, recipes, pantry inventory, meal planning, grocery generation, purchasing, and nutrition calculations in one system. The application uses FastAPI for typed HTTP services, Streamlit for the user experience, SQLAlchemy and Alembic for persistence, and a replaceable provider layer for USDA data and AI interpretation.

Area	Current capability	Evidence
Application	Household, nutrition, recipes, pantry, grocery, planning, and assistant workflows	End-to-end browser smoke tests
Agentic AI	Single preview-only orchestrator with typed intent and allowlisted deterministic tools	Phase 8A and 8B
Evaluation	49-case agent evaluation plus software regression testing	49/49 agent eval; 550 tests passed
External data	USDA FoodData Central search, import, normalization, cache, and provenance	Provider tests and migration 0003
Safety	Allergen, medical, prompt-injection, household-isolation, write, and concurrency controls	Guardrail and integrity tests
Deployment	Local Windows/uv workflow; SQLite development; PostgreSQL-compatible design	Production deployment remains pending

2 Product Vision and User Problem

Household food planning is fragmented. People maintain recipes in one place, groceries in another, and pantry information largely from memory. Nutrition targets rarely connect to actual ingredients, and food nearing expiration is discovered too late. NourishNest creates a continuous information flow from household needs to meal decisions and shopping actions.

The primary outcome is a practical answer to three everyday questions: What can we cook with what we have? Does it fit the household's needs? What exactly must we buy?

Design Principles

- Deterministic calculations are authoritative for calories, nutrients, quantities, conversions, inventory, and shortages.
- AI interprets language and coordinates approved tools; it does not invent operational facts.
- Household ownership is enforced at service and API boundaries.
- Potentially destructive or duplicate actions use confirmation, optimistic concurrency, idempotency, and database transactions.
- Incomplete or incompatible data produces explicit warnings rather than guessed conversions.
- The UI uses friendly names and hides technical identifiers under collapsed technical details.

3 Functional Capabilities

From Household Data to Actionable Planning

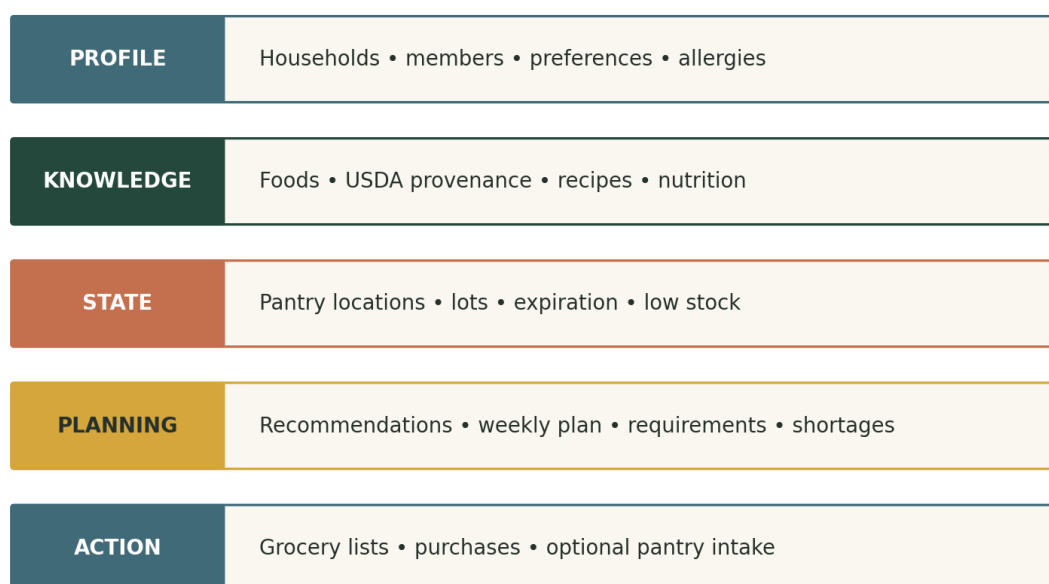


Figure 1. NourishNest capability layers

Domain	Implemented capability
Household	Create households and members; manage profile measurements, dietary preferences, and allergies; calculate saved-member nutrition.
Foods	Create and search normalized food records; import or refresh USDA FoodData Central items with provider-specific provenance.

Domain	Implemented capability
Recipes	Browse system and household recipes; create, edit, delete, view ingredients and instructions, and calculate deterministic nutrition.
Pantry	Track named locations and multiple lots; expiration, FEFO consumption, transfers, adjustments, discards, low-stock rules, and audit events.
Grocery	Create lists and items; preview recipe requirements and pantry shortages; generate lists with lineage; record purchases and optional pantry intake.
Planning	Rank recipes against pantry coverage and expiring stock; build session-based weekly plans; compare nutrition and preview shortages.
Assistant	Translate a natural-language meal-planning goal into typed constraints and a grounded, read-only plan preview.

4 System Architecture

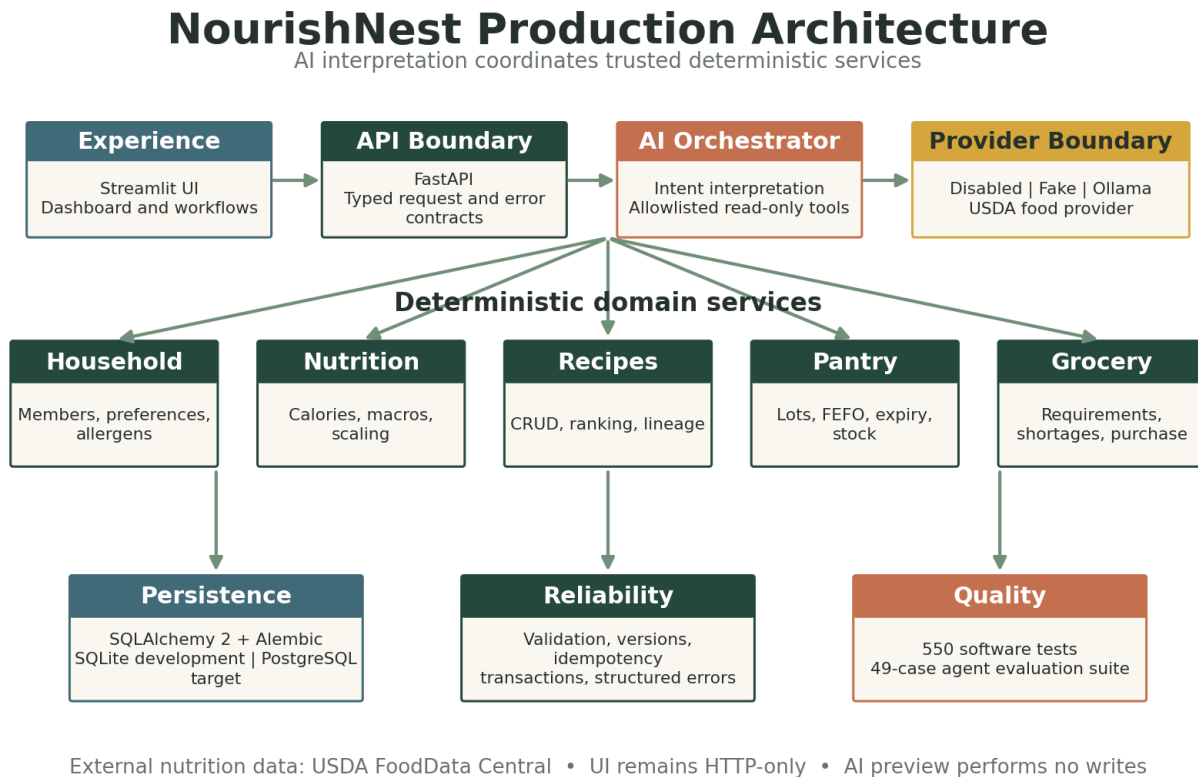


Figure 2. Layered application and AI architecture

4.1 Experience Layer

Streamlit provides the branded consumer interface. The UI calls FastAPI exclusively through a typed HTTP client; it does not import repositories or access the database. Navigation covers Dashboard, Household, Nutrition, Recipes, Pantry, Grocery Lists, Meal Planner, and AI Assistant. Session state retains active household, drafts, preview results, and retry keys where appropriate.

4.2 API and Service Layer

FastAPI exposes typed Pydantic contracts and a consistent error envelope containing code, message, and request ID. Services implement household isolation, validation, calculations, transaction boundaries, and orchestration. Repositories contain SQLAlchemy queries and provider-specific locking behavior.

4.3 Persistence Layer

SQLAlchemy 2 models use UUID primary keys, explicit timestamps, enumerated states, database constraints, relationship cascades, and NUMERIC fields for quantities and nutrition. Alembic manages incremental schema evolution. SQLite supports development and isolated tests; PostgreSQL is the intended production concurrency target.

4.4 Provider Boundaries

Food data and AI interpretation are replaceable interfaces. USDA FoodData Central supplies external nutrition data. The assistant provider supports disabled, deterministic fake, and optional local Ollama modes. Tests substitute fake providers and mocked HTTP transports, avoiding paid or live model calls.

5 Data Architecture and Migrations

Revision	Purpose	Primary schema contribution
0001	Household foundation	Households, members, dietary preferences, allergies
0002	Food and recipe foundation	Foods, allergens, tags, recipes, ingredients, instructions
0003	USDA provenance	Provider identity, external identifier, attribution, retrieval metadata
0004	Pantry	Locations, lots, stock rules, append-only transaction history
0005	Grocery foundation	Grocery lists and versioned grocery-list items
0006	Grocery generation	Generation runs, recipe lineage, generation references
0007	Purchase events	Purchase audit and optional pantry-intake linkage
0008	Member concurrency	Optimistic member version column and scoped mutation contract
0009	Recipe concurrency	Recipe versions and normalized creation idempotency keys

5.1 Core Entity Relationships

- A household owns members, pantry locations, pantry lots, stock rules, grocery lists, planning context, and household recipes.

- Foods are shared reference records and may carry manual, development-fixture, or external-provider provenance.
- Recipes contain ordered ingredients and instructions; ingredients reference foods and drive nutrition and grocery requirements.
- Pantry items represent lots, preserving location, quantity, canonical unit, expiration, status, and version.
- Grocery generation runs link selected recipes and generated items to auditable source records.
- Purchase events prevent duplicate intake and may atomically create a pantry lot and restock transaction.

5.2 Numeric Integrity

Quantities and nutrient values use Decimal in application code and NUMERIC(18,6) where precision is operationally important. Mass, volume, and count remain separate dimensions. The application converts compatible units into canonical grams, milliliters, or items and refuses to infer density.

6 Core Application Workflows

6.1 Nutrition

Member inputs are validated, converted into a shared nutrition profile, and processed by deterministic formulas. Recipe nutrition scales each ingredient through dimension-aware unit conversion, aggregates calories and macros, divides totals by servings, propagates allergens, and reports incomplete components as warnings.

6.2 Pantry Inventory

Pantry intake creates versioned lots. Consumption uses first-expiring-first-out selection across compatible active lots. Adjustments, transfers, discards, and restocks are atomic and append audit transactions. PostgreSQL queries use row locks where supported; optimistic versions reject stale writes. Idempotency keys prevent duplicated mutations.

6.3 Recipe Requirements and Shortages

A recipe selection is scaled to desired servings. Ingredient quantities are converted and aggregated by food and canonical dimension. Shortage preview subtracts active compatible pantry availability from requirements. Expired inventory is excluded, incompatible dimensions remain separate, and uncertain conversions become structured warnings.

6.4 Grocery Generation and Purchase

Generation atomically writes a run, positive-shortage items, recipe lineage, and a list-version update. Request fingerprints canonicalize recipe ordering and Decimal servings for idempotent replay. Purchases update item progress, completion state, list version, and purchase audit; optional pantry intake creates stock exactly once.

6.5 Recipe Recommendation

Recipe ranking is deterministic: 80 percent of the score reflects average quantity coverage and 20 percent reflects average coverage by expiring stock. Coverage is capped at one per required food and unit. Unsupported conversions score zero and emit warnings. Ties resolve by coverage, recipe name, and recipe ID.

7 Agentic AI Architecture

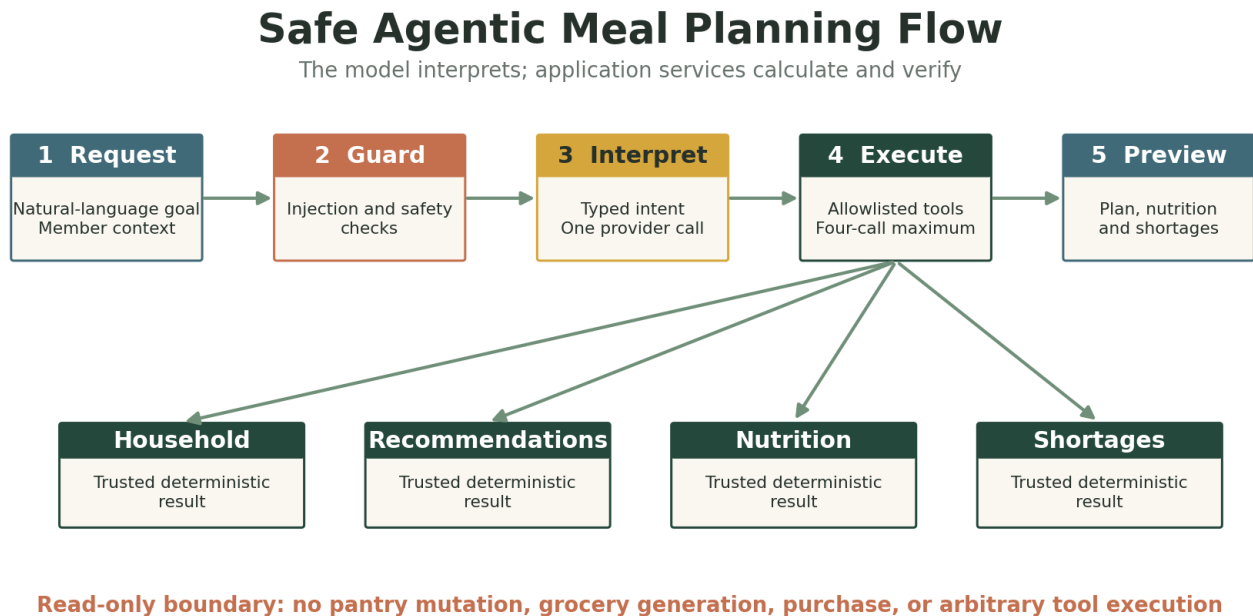


Figure 3. Controlled single-agent orchestration

7.1 What Makes the Assistant Agentic

The assistant accepts a goal expressed in natural language, interprets constraints into a typed intent, selects from a small allowlist of application tools, coordinates results, and returns a meal-plan preview. This is a bounded single-agent pattern: one interpretation step and no more than four tool calls.

7.2 Deterministic and Probabilistic Boundary

AI interpretation owns	Deterministic services own
Understanding requested cuisine, dietary pattern, time, calorie limit, meal count, and missing inputs	Household and member ownership

AI interpretation owns	Deterministic services own
Deciding which approved read-only capability is relevant	Recipe retrieval and scoring
Producing a concise conversational explanation from verified outputs	Nutrition, unit conversion, pantry availability, and grocery shortage arithmetic
Asking for clarification when the intent contract is incomplete	Database writes, versions, transactions, and idempotency

7.3 Provider Modes

Mode	Purpose	External model	Cost
disabled	Safe default	None	\$0
fake	Deterministic tests and local demonstrations	None	\$0
ollama	Optional local language-model interpretation	Local process	\$0 API cost

The Ollama adapter has strict JSON intent validation, bounded retries, input and output limits, and safe recovery messages. It was verified with mocked HTTP because Ollama and a local model were not installed. No claim is made about real-model accuracy or latency.

7.4 Guardrails

- Reject prompt injection attempts to override tool, safety, household, or preview constraints.
- Refuse diagnosis, treatment, guaranteed weight loss, and requests to ignore documented allergens.
- Validate typed model output and reject unknown fields, tools, and malformed responses.
- Run deterministic safety checks outside the language model before and after interpretation.
- Return a preview only; no pantry, grocery, recipe, purchase, or household mutation is available to the agent.
- Expose safe tool names, status, and duration without secrets or sensitive prompt content.

7.5 Why RAG and Multiple Agents Were Deferred

The current planning questions are answered by structured application data and deterministic services. A vector database would not improve exact pantry or nutrition arithmetic. Multiple agents would add coordination, latency, and failure modes without evidence that a single bounded orchestrator is insufficient. RAG becomes appropriate when NourishNest supports an authoritative document collection such as user recipes, cooking guidance, or curated nutrition references.

8 Evaluation and Quality Engineering

8.1 AI Evaluation Dataset

The versioned meal-planning evaluation suite contains 49 cases and runs locally with the fake provider. Coverage includes normal planning, vegetarian and vegan constraints, allergens, calorie and cooking-time limits, clarification, prompt injection, medical advice, guaranteed weight loss, provider errors, malformed output, and household isolation.

Metric	What it verifies
Intent extraction accuracy	Typed constraints match the requested meaning
Expected tool selection	Only the appropriate allowlisted tools run
Allergen safety	Documented allergens cannot be ignored
Dietary compliance	Returned recipes satisfy structured dietary constraints
Grounding	Recommendations reference recipes returned by deterministic retrieval
Numerical consistency	Displayed values match deterministic service results
Clarification correctness	Missing required inputs cause a useful question
Guardrail accuracy	Unsafe requests are refused consistently
Write-free behavior	Assistant preview does not change database state
Schema validity	Every response conforms to the typed contract

8.2 Verification Evidence

Checkpoint	Result
Phase 8A assistant service	453 tests passed; 49/49 AI evaluation cases passed
Phase 8B assistant UI	472 tests passed; isolated browser smoke passed

Checkpoint	Result
Phase 8C Ollama adapter	537 tests passed; mocked HTTP verification
Phase 9A visual redesign	550 tests passed; all checks passed
Static analysis	Ruff passed
Patch hygiene	git diff --check passed

The 49/49 score demonstrates contract and orchestration behavior under a deterministic test provider. It does not measure the quality of an untested real language model. The optional Ollama evaluation command records model, prompt, evaluation version, latency, metrics, and failures separately.

9 Security and Reliability

Control	Implementation
Tenant isolation	Household-scoped routes and ownership validation; inaccessible records return structured 404 responses
Input safety	Pydantic validation, strict enums, Decimal contracts, typed provider output
Concurrency	SQLAlchemy optimistic version columns and stale-write conflicts
Idempotency	Database-backed keys and canonical request fingerprints for mutation replay protection
Atomicity	Service-level transactions roll back inventory, grocery, lineage, and audit changes together
Auditability	Append-only pantry transactions, purchase events, generation runs, provenance, and request IDs
Provider resilience	Timeouts, bounded retries for safe reads, error classification, TTL caching, secret suppression
AI safety	Allowlisted tools, iteration limits, typed intent, prompt and response limits, preview-only execution

Current Security Gap

Authentication and end-user authorization are not yet implemented. Household scoping protects records within the current API contract, but there is no verified identity binding a caller to permitted households. This is the most important gap before public production deployment.

10 User Experience

The Streamlit experience was redesigned around a warm sage, cream, and terracotta brand. Navigation groups planning, shopping, management, and profile tasks. Friendly names replace UUID-heavy selectors; technical identifiers and request IDs appear only in collapsed details. Local WebP imagery, original SVG assets, responsive cards, professional empty states, and confirmation-gated destructive actions support a consumer-facing presentation.

Page	Primary experience
Dashboard	Household overview, expiring food, recommendations, weekly preview, quick actions
Household	Member cards, profile measurements, dietary preferences, allergies
Nutrition	Named member selection and server-calculated energy/macronutrient targets
Recipes	Visual browsing, detail, nutrition, household recipe editing
Pantry	Locations, lots, expiry, consume, adjust, transfer, discard, low stock
Grocery Lists	Requirement/shortage preview, generation, shopping progress, purchase intake
Meal Planner	Recommendations, seven-day slots, weekly nutrition, grocery preview
AI Assistant	Conversational request, clarification/refusal, grounded preview, technical trace

11 Phase History and Weekly Demonstration Guide

Phase	Theme	Demonstrable outcome
1	Data foundation	Households, members, preferences, allergies, FastAPI persistence
2	Foods and recipes	Recipe model, deterministic nutrition, unit conversion, seed fixtures
3	External food data	USDA provider, cache, retries, provenance, import and refresh
4	Pantry	Locations, lots, FEFO, expiration, low stock, integrity audit
5	Grocery lifecycle	CRUD, requirements, shortage, generation, lineage, purchase and pantry intake
6	Application UI	Streamlit shell and complete household, recipe, pantry, and grocery workflows
7	Smart planning	Deterministic recommendations and weekly meal-planning experience
8	Agentic AI	Single orchestrator, guardrails, evals, Assistant UI, optional Ollama adapter
9	Product polish	Brand system, responsive layouts, relevant local imagery, visual QA

Suggested Weekly Demo Narrative

Demo	Focus	What to show
Week 1	Problem and foundation	Show household creation, member preferences, architecture, and persistence.
Week 2	Nutrition and recipes	Create/search food, show recipe ingredients, deterministic conversions,

Demo	Focus	What to show
		and nutrition.
Week 3	Pantry intelligence	Add two lots, demonstrate expiry status, FEFO consumption, transfer, and low stock.
Week 4	Grocery automation	Scale recipes, preview requirements and shortages, generate a list, purchase with intake.
Week 5	Planning	Rank recipes by pantry and expiring-stock coverage; build a seven-day plan.
Week 6	Agent and evaluation	Submit a natural-language goal; show typed intent, tool trace, guardrail, and 49-case report.
Week 7	Product quality	Show branded responsive UI, test evidence, risks, and production roadmap.

12 Operations and Configuration

12.1 Local Startup

Run FastAPI and Streamlit from the repository root in separate PowerShell terminals.

```
# Terminal 1
$env:UV_LINK_MODE = "copy"
$env:APP_AI_PROVIDER = "fake"
uv run python -m uvicorn nourish_nest.api:app --host 127.0.0.1 --port 8000 --reload

# Terminal 2
$env:UV_LINK_MODE = "copy"
$env:APP_API_BASE_URL = "http://127.0.0.1:8000"
uv run python -m streamlit run streamlit_app.py --server.port 8501
```

12.2 Verification Commands

```
uv sync --extra dev
uv run alembic upgrade head
uv run python -m pytest
uv run ruff check .
git diff --check
uv run python -m nourish_nest.evals.meal_planning
```

12.3 Key Environment Configuration

Variable	Purpose	Typical local value
APP_DATABASE_URL	Database connection	SQLite default or PostgreSQL URL
APP_API_BASE_URL	Streamlit to FastAPI address	http://127.0.0.1:8000
APP_FOOD_DATA_PROVIDER	External food source	disabled or usda
USDA_API_KEY	FoodData Central credential	Secret environment value
APP_AI_PROVIDER	Assistant interpretation mode	disabled, fake, or ollama
APP_AI_MODEL	Optional local model name	Empty unless Ollama is used

Variable	Purpose	Typical local value
APP_OLLAMA_BASE_URL	Optional local inference endpoint	http://127.0.0.1:11434
APP_AI_TIMEOUT_SECONDS	Model/provider timeout	Configured bounded value

13 Limitations and Roadmap

13.1 Current Limitations

- Authentication, identity federation, role-based household membership, and authorization are not implemented.
- SQLite is appropriate for development but does not provide PostgreSQL-equivalent row-level locking or Decimal storage behavior.
- Meal plans and assistant conversations remain session-local and are not durable records.
- Pantry availability and recipe recommendations are point-in-time estimates without reservation of stock.
- The fake provider demonstrates orchestration, not open-ended language-model quality; Ollama was implemented but not deployed.
- Allergen matching uses normalized stored names without clinical synonym inference; nutrition output is informational, not medical advice.
- RAG, embeddings, chunking, vector search, semantic reranking, and multiple agents are not implemented.
- Existing generation rules limit one generation per grocery list; refund and purchase-reversal workflows are deferred.

13.2 Recommended Production Roadmap

Priority	Workstream	Outcome
P0	Identity and authorization	Managed authentication, user-to-household roles, token validation, endpoint policy tests
P0	Production data	PostgreSQL deployment, migration pipeline, backups, restore test, connection pooling
P1	Delivery	Container hardening, CI/CD, secret management, environment promotion, rollback
P1	Observability	Structured logs, readiness, metrics, traces, alerting, audit review
P1	Durable planning	Persisted meal plans and conversation summaries with privacy controls
P2	Real-model validation	Select an approved provider, run frozen evals, measure cost and latency, set

Priority	Workstream	Outcome
		release gates
P2	RAG	Only after defining authoritative documents, retrieval metrics, citations, and freshness rules
P3	Multiple agents	Only when eval evidence shows bounded single-agent orchestration is insufficient

Appendix A API Inventory

Domain	Representative route	Purpose
Health	GET /health	Service status and request ID
Households	GET/POST /v1/households	Discover and create households
Members	GET/POST /v1/households/{household_id}/members	Member collection
Members	GET/PUT/DELETE .../members/{member_id}	Scoped member operations and versions
Nutrition	POST /nutrition/calculate	Standalone deterministic target calculation
Foods	POST /v1/foods; GET /v1/foods/search	Stored-food creation and search
USDA	GET /v1/providers/usda/search and /foods/{fdc_id}	Provider search and detail
USDA	POST /v1/foods/import/usda/{fdc_id}	Import and refresh external food
Recipes	CRUD /v1/households/{household_id}/recipes	Household/system recipe workflows
Recipes	GET .../recipes/{recipe_id}/nutrition	Deterministic recipe nutrition
Pantry	.../pantry/locations and .../pantry/items	Locations and lot inventory

Domain	Representative route	Purpose
Pantry	.../consume, adjust, transfer, discard	Atomic stock mutations
Pantry	.../expiring, expired, low-stock, summary	Inventory intelligence
Grocery	CRUD .../grocery-lists and items	List and manual-item lifecycle
Requirements	POST .../grocery-requirements/preview	Scaled recipe needs
Shortage	POST .../grocery-requirements/shortage-preview	Pantry-aware purchase needs
Generation	POST .../grocery-lists/{list_id}/generations	Idempotent generated list
Purchase	POST .../items/{item_id}/purchase	Audit and optional pantry intake
Planning	POST .../recipe-recommendations	Deterministic ranked recipes
Assistant	POST .../assistant/meal-plan-preview	Read-only agentic preview

Appendix B Demonstration Script

Step	Action	Talking point
1	Open Dashboard	Introduce the household food-management problem and show the professional shell.
2	Choose Household	Show named household and member context, preferences, and allergens.
3	Inspect Pantry	Show locations, two lots, expiry status, low stock, and FEFO behavior.
4	Browse Recipes	Open a recipe, ingredients, instructions, nutrition, and pantry-match rationale.
5	Plan Week	Add recommended recipes to days and display weekly nutrition and shortages.
6	Ask Assistant	Submit: Plan five vegetarian dinners under 600 calories and use food expiring soon.
7	Explain Agent	Open technical details and show typed constraints, safe tool trace, versions, and warnings.
8	Show Guardrail	Try a guaranteed-weight-loss or ignore-allergies request and show the refusal.
9	Show Evaluation	Run or display the 49/49 fake-provider evaluation report and explain its scope.
10	Close	Summarize deterministic grounding, current limitations, and production roadmap.

Suggested Opening

NourishNest is an AI-assisted household nutrition and food-management application. It connects member needs, recipes, pantry inventory, meal planning, and grocery shopping. The key technical

idea is that the AI understands the user's goal and coordinates trusted tools, while deterministic services remain responsible for nutrition, quantities, inventory, and safety-critical results.

Suggested Closing

The project demonstrates more than a chat interface. It combines a production-oriented domain model, provider abstractions, transactional integrity, guarded agent orchestration, a versioned evaluation dataset, and a polished user experience. The next production priorities are authentication, PostgreSQL deployment, CI/CD, secrets, and observability; RAG and multiple agents will be added only when a defined use case and evaluation evidence justify them.